

1 Table of Contents

- Introduction
 - Purpose
 - Scope
 - Definitions, acronyms and abbreviations
 - References
 - Overview
- Overall description
 - Product Perspective
 - Product Functions
 - User Characteristics
 - Constraints
 - Assumptions and Dependencies
- Specific Requirements
 - Functional Requirements
 - * Maze Generation and Display
 - * Player Controls
 - * Ai Rival
 - * Game Logic
 - Non Functional Requirements

2 Introduction

This document provides the foundation for the *Memory Maze - Human vs AI Pathfinding Game* project.

2.1 Purpose

This document specifies the requirements for the *Memory Maze - Human vs AI Pathfinding Game*, where a human player competes against an AI rival to solve dynamically generated mazes. It is intended for developers and supervisors involved in designing, implementing, and testing the system.

2.2 Scope

Memory Maze - Human vs AI Pathfinding Game aims to develop a competitive maze-solving game where a human player competes against an AI rival. At the beginning, the maze is displayed fully for a few seconds, after which parts of it disappear. The player must rely on memory and strategy to find the exit. Simultaneously, the AI rival uses the A* Search Algorithm (Manhattan Distance heuristic) to navigate the maze efficiently.

The game tests human cognitive skills against algorithmic precision, providing engaging experience and demonstrating pathfinding, algorithm design, and AI vs human performance comparison while keeping track of results. Additional complexity is introduced by multiple maze sizes, increasing difficulty levels, and power-ups, which can hinder or help in progress.

2.3 Definitions, Acronyms and Abbreviations

- UC: User Characteristic (e.g. UC1 means `user characteristic 1`)
- FR: Functional Requirement (e.g. FR1 means `functional requirement 1`)
- NFR: Non Functional Requirement (e.g. NFR1 means `non functional requirement 1`)
- A*: Path finding algorithm
- Manhattan distance heuristic: A calculation useful in grid based pathfinding algorithms
- Python: Programming language
- PyGame: A library which integrates with Python

2.4 References

- IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1993, IEEE Computer Society, 1993. Available: <https://ieeexplore.ieee.org/>

2.5 Overview

This document describes the

- Product features
- Models
- User Characteristics
- Constraints
- Assumptions
- Functional requirements

3 Overall Description

3.1 Product Perspective

The *Memory Maze - Human vs AI Pathfinding Game* is a standalone desktop application built using Python and the PyGame library. It integrates maze generation, AI pathfinding, and real-time gameplay in a single environment.

3.2 Product Functions

- Generate and display random mazes.
- Run player and AI navigation concurrently.
- Handle player controls and power-ups.
- Track scores, levels, and performance metrics.
- Manage game states (pause, restart, exit).

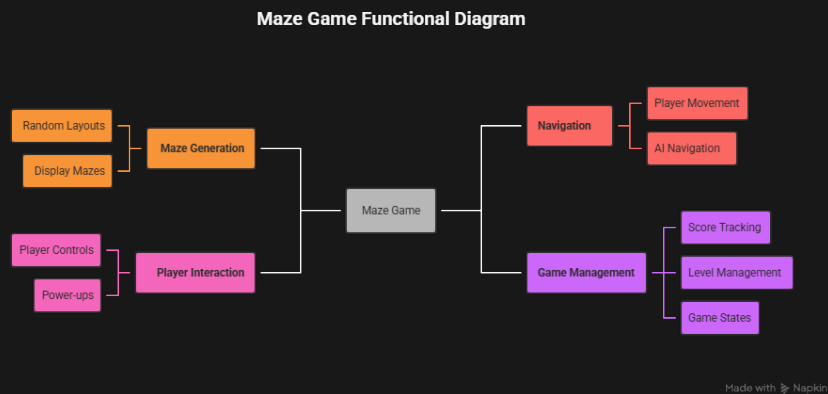


Figure 1: Maze Game Functional Diagram

3.3 User Characteristics

- Player
 - UC1: Basic familiarity with keyboard controls and maze logic.
- Developer
 - UC2: Should understand Python and PyGame library.
- Supervisor
 - UC3: Evaluates performance and functionality.

3.4 Constraints

- Developed using Python and PyGame only.
- Runs on desktop
 - Windows
- Algorithm(s) for maze generation
 - Recursive Backtracking

- Prim's Algorithm
- Algorithm(s) for path finding
 - A* with Manhattan Distance heuristic
- Maze difficulties
 - Small
 - Medium
 - Large

3.5 Assumptions and Dependencies

- Adequate hardware resources for real-time graphics rendering.
 - Monitor
 - Graphics Card
- Operating system should be installed
 - Windows
- Python 3.13.0+ and PyGame are installed.
- Single-player mode only (one human vs. one AI).

4 Specific Requirements

4.1 Functional Requirements

4.1.1 Maze Generation and Display

- FR1: The system shall generate random mazes using `recursive backtracking` or `Prim's` algorithm.
- FR2: The maze shall be fully visible for a defined time (e.g., 5–10 seconds).
- FR3: After the reveal time, walls shall disappear partially, requiring the player to rely on memory.
- FR4: Maze complexity (size and branching factor) shall increase with levels.

4.1.2 Player Controls

- FR5: The player shall move using keyboard arrow keys.
- FR6: The player shall be allowed to collect power-ups (e.g., reveal part of maze, slow down AI).
- FR7: The player shall be penalized for hitting dead ends (e.g., time penalty).

4.1.3 Ai Rival

- FR8: The AI rival shall use the **A*** Search Algorithm with **Manhattan Distance heuristic** to compute its shortest path.
- FR9: The AI shall update its path dynamically if obstacles appear or change.
- FR10: The AI's speed shall increase with higher levels to maintain difficulty.

4.1.4 Game Logic

- FR11: The game shall run both player and AI simultaneously, updating positions in real-time.
- FR12: The game shall declare a winner when either player or AI reaches the exit first.
- FR13: The game shall maintain a scoreboard (wins, losses, fastest completion times).
- FR14: The game shall support multiple difficulty levels (small, medium, large mazes).
- FR15: The system shall allow the player to restart, pause, and exit at any time.

4.2 Non Functional Requirements

- NFA1: Frames per second should be at least 30.